A Variational Approach to Encrypted Model Predictive Control

Jihoon Suh, Yeongjun Jang, Junsoo Kim, and Takashi Tanaka

***Abstract*— We present a novel variational approach to encrypted model predictive control (VEMPC) as a solution to privacy-preserving model predictive control. VEMPC has algorithmic advantage to solving encrypted MPC compare to existing approaches.**

***Index Terms*— Encrypted Control, Fully Homomorphic Encryption, Model Predictive Control, Variational Approach**

I. Tasks

Priorities alphanumerically.

- **(A-1)** Numerical Experiment (make it work without λ making the $\tilde{\kappa}$ degenerate.
  - Let's fix polynomial approximation method.
- **(A-2)** $\hat{U}_{K,p}$ estimator error analysis (Section V-F)
- **(B-1)** Literature review
- **(B-2)** Consider changes in title ("variational" term may be a bit ambiguous and can mean different things)
- (**Future Work**) Mitigate λ and $\tilde{\kappa}$ coupling issue $\rightarrow$ poor sampling (covariance shrinks) & mean shifting away from feasible set

Literature survey on deterministic MPPI

- [1]
  - Investigates suboptimality of MPPI to deterministic optimal control problems
  - Quantifies deviation between control provided by MPPI and the optimal solution to the deterministic optimal control problem
  - In the introduction, says that MPPI has often been used for deterministic optimal control problems [2]–[6]
  - In particular, see Section III-C. In equation (20), it is shown that the solution provided by MPPI converges to the optimal solution to the deterministic optimal control problem as variance of the injected noise and λ goes to zero. This part and Theorem 1 seems most relevant for our setting.
  - Precisely,
- [6]
  - Provides a PI based approach for solving deterministic discrete time optimal control problems with finite horizon and signal temporal logic (STL) costs
  - Obtains an unbiased solution for the deterministic solution by gradually reducing Σ (covariance matrix of sampling noise) and λ (constant used for exponential transformation). Also see Line 2 of Algorithm 1 in [1], which employs a similar method.

J. Suh, and T. Tanaka are with the School of Aeronautics and Astronautics, Purdue University, West Lafayette, IN 47907, USA (e-mail: {suh95, tanaka16}@purdue.edu).

II. Introduction

A. Motivation

MODEL Predictive Control (MPC) is a widely adopted control paradigm across industries... As MPC relies heavily on modern optimization algorithms, third party compute is often an attractive solution to scale... However, the knowledge of model, state/input trajectories, as well as constraint information are often privacy-sensitive. For this reason, there has been recent interests in developing privacy-preserving algorithms for MPC (cite Encrypted MPC works).

Existing encrypted MPC has been primarily focusing on directly adopting standard algorithms to solve the given QP. These include various gradient methods, explicit MPC..., and .... However, they often necessitate intermediate decryption, computationally inefficient, or complex communication architecture.

We design a conceptually simple encrypted MPC architecture, realized by approximate, randomized algorithm via adopting variational framework.

Variational approaches are ubiquitous in many recent algorithms, e.g., D, E, F.

Surprising, we find also that variational approach is a convenient framework for Encrypted MPC.

B. Main Contributions

This paper makes X main contributions:

- variational encrypted model predictive control (VEMPC).
- simple architecture
- online efficiency

Notation: The sets of integers and complex numbers are denoted by $\mathbb{Z}$ and $\mathbb{C}$, respectively. For a vector $x \in \mathbb{C}^n$, $\|x\|$ denotes the infinity norm, and $\odot$ represents the Hadamard (element-wise) product. We denote by $\mathsf{RotVec}_r(x)$ the cyclic upward shift of x by r positions (e.g., $\mathsf{RotVec}_1([1,2,3,4]^\top) = [2,3,4,1]^\top$).

III. Preliminaries

A. CKKS Cryptosystem

We briefly review the Cheon-Kim-Kim-Song (CKKS) cryptosystem [7], which is a fully homomorphic encryption scheme that enables direct evaluation of additions and multiplications on encrypted data.

The CKKS scheme consists of algorithms KeyGen, Enc, and Dec. Given a security parameter $\lambda \in \mathbb{N}$, the key generation algorithm KeyGen outputs a secret key sk. For a plaintext $m \in \mathbb{C}^{N/2}$, where $N \in \mathbb{N}$, the encryption algorithm takes sk and outputs a ciphertext $\mathsf{Enc}(m) \in \mathcal{C}$ with $\mathcal{C}$ denoting the ciphertext space. The decryption algorithm uses sk to approximately recover the underlying plaintext, as $\mathsf{Dec}(\mathsf{Enc}(m)) \approx m$. The magnitude of the approximation error is primarily governed by the scaling factor $\Delta > 0$ used to encode the plaintext through scaling and rounding during encryption, and the choice of N.

The described scheme supports homomorphic addition, multiplication, and vector rotation. These operations are represented, respectively, by

$$\oplus : \mathcal{C} \times \mathcal{C} \to \mathcal{C} \qquad \otimes : \mathcal{C} \times \mathcal{C} \to \mathcal{C}, \qquad \mathsf{RotCt}_r : \mathcal{C} \to \mathcal{C}.$$

Importantly, the inherent approximation error in a ciphertext accumulates under these operations, and thus, the errors need to be refreshed periodically to prevent decryption failure. This is achieved via the *bootstrapping* operation $\mathsf{Boot} : \mathcal{C} \to \mathcal{C}$, which resets the accumulated error and enables an arbitrary number of sequential homomorphic operations [8].

The following lemma summarizes the grwoth of errors induced by the above homomorphic operations. For conciseness, the bounds are state using $\mathcal{O}$-notation and concrete derivations can be found in [7], [8].

Lemma 1: For any $x \in \mathbb{C}^{N/2}$, $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, and $r \geq 0$, there exist $\mathsf{B}^{\mathsf{Enc}}, \mathsf{B}^{\mathsf{Mult}}, \mathsf{B}^{\mathsf{RotCt}}, \mathsf{B}^{\mathsf{Boot}} \in \mathcal{O}(N/\Delta)$ such that

$$\|\mathsf{Dec}(\mathsf{Enc}(x)) - x\| \leq \mathsf{B}^{\mathsf{Enc}}, \tag{1a}$$

$$\|\mathsf{Dec}(\mathbf{c} \oplus \mathbf{c}') - (\mathsf{Dec}(\mathbf{c}) + \mathsf{Dec}(\mathbf{c}'))\| = 0, \tag{1b}$$

$$\|\mathsf{Dec}(\mathbf{c} \otimes \mathbf{c}') - \mathsf{Dec}(\mathbf{c}) \odot \mathsf{Dec}(\mathbf{c}')\| \leq \mathsf{B}^{\mathsf{Mult}}, \tag{1c}$$

$$\|\mathsf{Dec}(\mathsf{RotCt}_r(\mathbf{c})) - \mathsf{RotVec}_r(\mathsf{Dec}(\mathbf{c}))\| \leq \mathsf{B}^{\mathsf{RotCt}}, \tag{1d}$$

$$\|\mathsf{Dec}(\mathsf{Boot}(\mathbf{c})) - \mathsf{Dec}(\mathbf{c})\| \leq \mathsf{B}^{\mathsf{Boot}}. \tag{1e}$$

In fact, the parameters N and Δ affect not only the error growth in Lemma 1 but also the security level of the scheme. Typically, increasing N or decreasing Δ strengthens security, potentially at the cost of higher computational cost. In this letter, we assume that these parameters are given based on standard security requirements, and focus on integrating the CKKS scheme into our control framework. For detailed parameter selection methods, we refer the reader to [7].

B. Model Predictive Control (MPC)

MPC [9] is a receding-horizon control strategy in which, at each time step, a finite-horizon optimization problem is solved and the first element of the optimal control sequence is applied. We consider a deterministic, linear-quadratic (LQ) setup with linear dynamics and a quadratic cost.

At time $t = 0$, the controller solves a N-horizon optimization problem. Let $X_0 := \{x_k\}_{k=0}^{N}$ and $U_0 := \{u_k\}_{k=0}^{N-1}$ denote the predicted state and input sequences, where x_0 is the current measured state. The quadratic cost to be minimized is

$$J_0(x_0, U_0) := x_N^\top Q_f x_N + \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k\right), \tag{2}$$

where $Q_f, Q \in \mathbb{R}^{n \times n}$ are positive semidefinite and $R \in \mathbb{R}^{m \times m}$ is positive definite.

The state and input are subject to polyhedral constraints $\mathcal{X} := \{x \in \mathbb{R}^n : G_x x \leq h_x\}$ and $\mathcal{U} := \{u \in \mathbb{R}^m : G_u u \leq h_u\}$. The MPC problem is then formulated as

$$\begin{aligned} \min_{U_0} \quad & J_0(x_0, U_0) \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k, \quad k = 0, \dots, N-1, \\ & x_k \in \mathcal{X}, \quad k = 1, \dots, N, \\ & u_k \in \mathcal{U}, \quad k = 0, \dots, N-1, \\ & x_0 \text{ given}, \end{aligned} \tag{3}$$

which is a quadratic program (QP) with linear inequality constraints. Standard solution approaches include implicit MPC, where the QP is solved online at each time step, and explicit MPC, where a parametric feedback law is pre-synthesized offline for a more efficient online implementation.

C. Variational Formula

Consider a reference probability distribution κ_0 on $\mathbb{R}^d$. We say that a distribution κ is absolutely continuous with respect to κ_0, denoted $\kappa \ll \kappa_0$, if there exists a nonnegative measurable function $\frac{d\kappa}{d\kappa_0}$, the Radon-Nikodym derivative of κ with respect to κ_0, such that

$$\kappa(du) = \frac{d\kappa}{d\kappa_0}(u)\, \kappa_0(du). \tag{4}$$

Let $C : \mathbb{R}^d \to \mathbb{R}$ be a measurable cost function and let $\lambda > 0$. We consider the following convex optimization problem over the set of probability distributions

$$\min_{\kappa} \ \mathbb{E}^{\kappa}[C(u)] + \lambda\, D(\kappa \| \kappa_0), \tag{5}$$

where the Kullback-Leibler (KL) divergence is defined as

$$D(\kappa \| \kappa_0) := \int \log\left(\frac{d\kappa}{d\kappa_0}(u)\right) \kappa(du), \tag{6}$$

when $\kappa \ll \kappa_0$, and $D(\kappa \| \kappa_0) = +\infty$ otherwise.

The minimizer κ^* is unique since the objective in (5) is strictly convex. Moreover, the minimizer κ^* satisfies

$$\frac{d\kappa^*}{d\kappa_0}(u) = \frac{\exp(-C(u)/\lambda)}{\int \exp(-C(v)/\lambda)\, \kappa_0(dv)}. \tag{7}$$

The minimum value of (5) admits the explicit expression

$$-\lambda \log \mathbb{E}^{\kappa_0}\left[\exp\left(-\frac{C(u)}{\lambda}\right)\right], \tag{8}$$

which is commonly referred to as the free energy.

A point estimate for the optimal distribution κ^* is

$$\hat{u} := \mathbb{E}^{\kappa^*}[u] = \frac{\mathbb{E}^{\kappa_0}[u \exp(-C(u)/\lambda)]}{\mathbb{E}^{\kappa_0}[\exp(-C(u)/\lambda)]}. \tag{9}$$

As $\lambda \to 0$, the distribution κ^* concentrates on the minimizers of C, and $\hat{u}$ converges to a minimizer of C.

IV. Problem Formulation

We first formulate the encrypted model predictive control problem so that it can be solved using the variational approach.

The following subsections describe several reformulations of LQ-MPC (3) towards the equivalent unconstrained optimization after which the variational approach is invoked.

A. Compact Representation of MPC

Let $U := [u_0^\top, u_1^\top, \dots, u_{N-1}^\top]^\top \in \mathbb{R}^{Nm}$ and $X := [x_1^\top, x_2^\top, \dots, x_N^\top]^\top \in \mathbb{R}^{Nn}$ denote the stacked sequences of control inputs and predicted states, respectively. By substituting dynamics constraints, the state trajectory is described as a linear equation of the initial state x_0 and control inputs U

$$X = \Lambda x_0 + \Psi U, \tag{10}$$

where $\Lambda \in \mathbb{R}^{Nn\times n}$ and $\Psi \in \mathbb{R}^{Nn\times Nm}$ are given by

$$\Lambda := \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \quad \Psi := \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}. \tag{11}$$

The block diagonal matrices $\bar{Q} = \operatorname{diag}(Q, \dots, Q, Q_f) \in \mathbb{R}^{Nn\times Nn}$ and $\bar{R} = \operatorname{diag}(R, \dots, R) \in \mathbb{R}^{Nm\times Nm}$ can be defined such that the quadratic cost is as follows

$$J_0(x_0, U) := x_0^\top Q x_0 + X^\top \bar{Q} X + U^\top \bar{R} U. \tag{12}$$

Substituting (10), the quadratic cost can be written as

$$J_0(x_0, U) = x_0^\top P x_0 + x_0^\top S U + \frac{1}{2} U^\top H U, \tag{13}$$

where

$$\begin{aligned} P &:= Q + \Lambda^\top \bar{Q} \Lambda \in \mathbb{R}^{n\times n}, \\ S &:= 2\Lambda^\top \bar{Q} \Psi \in \mathbb{R}^{n\times Nm}, \\ H &:= 2\big(\bar{R} + \Psi^\top \bar{Q} \Psi\big) \in \mathbb{R}^{Nm\times Nm}. \end{aligned} \tag{14}$$

With general polyhedral constraints on the states and inputs, the LQ-MPC (3) is equivalent to the following QP

$$\begin{aligned} \min_{U\in\mathbb{R}^{Nm}} \quad & x_0^\top P x_0 + x_0^\top S U + \frac{1}{2} U^\top H U && \text{(15a)} \\ \text{subject to} \quad & \bar{G}_x(\Lambda x_0 + \Psi U) \le \bar{h}_x, && \text{(15b)} \\ & \bar{G}_u U \le \bar{h}_u. && \text{(15c)} \end{aligned}$$

Here $\bar{G}_x := \operatorname{diag}(G_x, \dots, G_x) \in \mathbb{R}^{Np_x\times Nn}$, $\bar{h}_x := [h_x^\top, \dots, h_x^\top]^\top \in \mathbb{R}^{Np_x}$, $\bar{G}_u := \operatorname{diag}(G_u, \dots, G_u) \in \mathbb{R}^{Np_u\times Nm}$, and $\bar{h}_u := [h_u^\top, \dots, h_u^\top]^\top \in \mathbb{R}^{Np_u}$.

B. Desirability and Feasibility

More compactly, (15b) and (15c) define

$$\begin{aligned} G &:= \begin{bmatrix} \bar{G}_x \Psi \\ \bar{G}_u \end{bmatrix} \in \mathbb{R}^{p\times Nm} \\ h(x_0) &:= \begin{bmatrix} \bar{h}_x - \bar{G}_x \Lambda x_0 \\ \bar{h}_u \end{bmatrix} \in \mathbb{R}^{p}, \end{aligned} \tag{16}$$

where p is the number of inequality constraints. Then, the feasible set can be defined using $g(U; x_0) := GU - h(x_0)$

$$\mathcal{F}(x_0) := \{U \in \mathbb{R}^{Nm} : g(U; x_0) \le 0\}. \tag{17}$$

What we want to achieve is to assign higher desirability to the trajectories that live inside the feasible set $\mathcal{F}(x_0)$.

One simple way to model this behavior is by defining the desirability (for the feasible set) as follows

$$r(U; x_0) := \mathbf{1}_{\mathcal{F}(x_0)}(U). \tag{18}$$

Consider the corresponding indicator function

$$\phi(U; x_0) := \begin{cases} 0, & U \in \mathcal{F}(x_0), \\ +\infty, & \text{otherwise.} \end{cases} \tag{19}$$

By including this hard penalty indicator ϕ, we can now reformulate (15) as an unconstrained problem, as follows

$$\min_{U\in\mathbb{R}^{Nm}} \ \mathcal{J}(U; x_0). \tag{20}$$

The extended value functional $\mathcal{J}(U; x_0) := J_0(x_0, U) + \phi(U; x_0)$ encodes the desirability through the indicator function as $\exp(-\phi(U; x_0)/\lambda) = \mathbf{1}_{\mathcal{F}(x_0)}(U)$ for all $\lambda > 0$.

In the subsequent sections, we replace the desirability by an HE-friendly polynomial surrogate.

V. A Variational Approach to Encrypted MPC

A. Variational Reformulation with Feasibility Indicator

Let κ_0 be a reference probability distribution on $\mathbb{R}^{Nm}$ and let $\lambda > 0$. We now consider the variational version over probability distributions κ on $\mathbb{R}^{Nm}$, that is, we solve

$$\min_{\kappa} \ \mathbb{E}^{\kappa}[\mathcal{J}(U; x_0)] + \lambda\, D(\kappa \| \kappa_0). \tag{21}$$

The optimizer satisfies $\kappa^* \ll \kappa_0$. Moreover, the variational problem (21) admits a unique minimizer κ^*, and it satisfies

$$\frac{d\kappa^*}{d\kappa_0}(U) = \frac{\exp(-\mathcal{J}(U; x_0)/\lambda)}{\int \exp(-\mathcal{J}(V; x_0)/\lambda)\, \kappa_0(dV)}. \tag{22}$$

The variational control sequence can be estimated by

$$\hat{U}(x_0) := \mathbb{E}^{\kappa^*}[U] = \frac{\mathbb{E}^{\kappa_0}\left[U \, \exp\left(-\frac{\mathcal{J}(U;x_0)}{\lambda}\right)\right]}{\mathbb{E}^{\kappa_0}\left[\exp\left(-\frac{\mathcal{J}(U;x_0)}{\lambda}\right)\right]}. \tag{23}$$

It is a ratio of expectations under the reference measure κ_0, and the Monte-Carlo method can be used to estimate.

Infeasible trajectories are undesirable and receive zero weight, i.e., $\exp\left(-\frac{\mathcal{J}(U;x_0)}{\lambda}\right) = \exp\left(-\frac{J_0(x_0,U)}{\lambda}\right) r(U; x_0)$.

B. Compute Architecture and Role Division

Need to be re-written according to latest.

C. Reference Distribution

The performance of computing (23) depends on the choice of reference distribution κ_0. This dependence becomes more pronounced in the context of encrypted MPC, where the reference distribution must satisfy additional encrypted computational constraints. In our protcol, this means that the reference distribution must be easy (within encrypted computation domain) to sample from, and also the importance sampling weights should be efficiently evaluated.

We start with the reference distribution κ_0 as a passive open-loop Gaussian distribution on $\mathbb{R}^{Nm}$. Specifically, we define

$$\kappa_0 := \mathcal{N}(0, \Sigma_0), \tag{24}$$

where $\Sigma_0 \succ 0$ is a user-chosen covariance matrix.

Incorporate LQR after change in constraint formulation

D. Exponential Tilting of Gaussian κ_0

For fixed x_0, the penalized cost decomposes as

$$\mathcal{J}(U; x_0) = J_0(U; x_0) + \phi(U; x_0), \tag{25}$$

where the quadratic term $J_0(U; x_0)$ follows (13).

Let $\kappa_0 = \mathcal{N}(0, \Sigma_0)$. Completing the square yields an exponentially tilted Gaussian distribution

$$\tilde{\kappa}(\cdot \mid x_0) := \mathcal{N}(m_U(x_0), \Sigma_U), \tag{26a}$$

$$\Sigma_U := \left(\Sigma_0^{-1} + \frac{1}{\lambda} H\right)^{-1}, \tag{26b}$$

$$m_U(x_0) := -\frac{1}{\lambda} \Sigma_U S^\top x_0. \tag{26c}$$

This construction absorbs the quadratic cost J_0 into the sampling distribution, leaving only the feasibility term $\phi(U; x_0)$ to be handled by importance sampling.

Lemma 2: Let $\kappa_0 = \mathcal{N}(0, \Sigma_0)$ and $\tilde{\kappa}(\cdot \mid x_0)$ be defined by (26). Then there exists a constant $c_Q(x_0) > 0$, independent of U, such that

$$\exp\left(-\tfrac{\mathcal{J}(U;x_0)}{\lambda}\right)\kappa_0(dU) = c_Q(x_0)\, r(U; x_0)\, \tilde{\kappa}(dU \mid x_0).$$

Proof: The result follows by substituting $J_0(U; x_0) = \frac{1}{2} U^\top H U + (S^\top x_0)^\top U + c(x_0)$ into $\exp(-J_0/\lambda)\kappa_0(dU)$ and completing the square to identify the Gaussian measure $\tilde{\kappa}(\cdot \mid x_0)$. All terms independent of U are absorbed into $c_Q(x_0)$. ■

The constant $c_Q(x_0)$ collects all terms independent of U, including $\exp(-c(x_0)/\lambda)$ and the Gaussian normalization term.

E. Encrypted Monte-Carlo Estimator

In the encrypted client-cloud architecture of Section V-B, the cloud generates randomized samples and evaluates encrypted penalty values, while all sensitive quantities remain encrypted.

1) Sampling distribution: Since $\tilde{\kappa}(\cdot \mid x_0)$ is Gaussian, samples can be generated as

$$U^{(i)} = m_U(x_0) + L_U \xi^{(i)}, \qquad \xi^{(i)} \sim \mathcal{N}(0, I), \tag{27}$$

where $\Sigma_U = L_U L_U^\top$.

2) Encrypted sampling protocol under $\tilde{\kappa}$: Let $\Sigma_U = L_U L_U^\top$ be a Cholesky factorization with $L_U \in \mathbb{R}^{Nm \times Nm}$. The client encrypts and transmits an HE-compatible representation of L_U offline. The cloud can then precompute a bank of encrypted Gaussian perturbations $\{\mathsf{Enc}(L_U \xi^{(i)})\}_{i=1}^K$, where $\xi^{(i)} \sim \mathcal{N}(0, I)$ are sampled in plaintext. Otherwise, this linear transformation would be prohibitive for online execution.

During the online phase, the client transmits only the encrypted mean $\mathsf{Enc}(m_U(x_0))$, and the online task for the cloud reduces to encrypted additions, which is relatively cheap to execute. As a result, the cloud effectively forms the encrypted control samples equivalent to the operation below

$$\mathsf{Enc}(U^{(i)}) := \mathsf{Enc}(m_U(x_0)) \oplus \mathsf{Enc}(L_U \xi^{(i)}). \tag{28}$$

Upon decryption, $\{U^{(i)}\}_{i=1}^K$ are i.i.d. samples from $\tilde{\kappa}(\cdot \mid x_0)$.

3) Importance weights and estimator: By Lemma 2, the variational estimate (23) can equivalently be written as

$$\hat{U}(x_0) = \frac{\mathbb{E}^{\tilde{\kappa}(\cdot|x_0)}[U\, r(U; x_0)]}{\mathbb{E}^{\tilde{\kappa}(\cdot|x_0)}[r(U; x_0)]}. \tag{29}$$

Consequently, the Monte Carlo estimator

$$\hat{U}_K(x_0) := \frac{\sum_{i=1}^K U^{(i)} r(U^{(i)}; x_0)}{\sum_{i=1}^K r(U^{(i)}; x_0)}. \tag{30}$$

approximates the variational solution $\hat{U}(x_0)$.

Proposition 1: Assume fixed (x_0). Let $\{U^{(i)}\}_{i=1}^K$ be i.i.d. samples from $\tilde{\kappa}(\cdot \mid x_0)$. Then $\hat{U}_K(x_0) \to \hat{U}(x_0)$ almost surely as $K \to \infty$.

Proof: SLLN, and almost sure convergence. ■

The variational solution $\hat{U}(x_0) \to U^*(x_0)$ as $\lambda \to 0$.

F. Polynomial Surrogate of the Penalty Cost

Our proposed protocol for computing (23) reduces the (encrypted) computation complexity by absorbing the quadratic cost into the sampling process. However, the desirability for the feasibility is not directly computable under homomorphic encryption. We therefore introduce a polynomial surrogate.

1) Penalty Structure and Approximation: Recall that the feasible set is defined through the residual

$$g(U; x_0) = GU - h(x_0). \tag{31}$$

Feasibility requires $g_j(U; x_0) \le 0$ for all $j = 1, \cdots, p$.

One way to quantify the constraint violation is by using the positive part (often known as ReLU),

$$[g_j(U; x_0)]_+ := \max\{g_j(U; x_0), 0\}. \tag{32}$$

Define the aggregate violation score

$$s(U; x_0) := \sum_{j=1}^{p} [g_j(U; x_0)]_+. \tag{33}$$

Since the nonlinearity $[\cdot]_+$ is not directly computable under homomorphic encryption, we approximate it by a degree-ℓ polynomial.

2) Offline Polynomial Design: We design our polynomial approximation for a bound $B_\ell > 0$ in the following steps.

First, on the normalized interval $[-1, 1]$, we construct a degree-ℓ Chebyshev polynomial for the function $\rho \mapsto B_\ell[\rho]_+$

$$\hat{h}_\ell(\rho) = \sum_{k=0}^{\ell} a_k \rho^k. \tag{34}$$

We require the following uniform approximation error to hold for the polynomial

$$\delta_\ell := \sup_{\rho \in [-1,1]} \left| B_\ell[\rho]_+ - \hat{h}_\ell(\rho) \right| \tag{35}$$

The normalization for the argument of the polynomial will be absorbed into the polynomial coefficients. That is, we define

$$c_k := \frac{a_k}{B_\ell^k}, \qquad k = 0, \dots, \ell. \tag{36}$$

This defines the polynomial

$$h_\ell(\gamma) := \sum_{k=0}^{\ell} c_k \gamma^k, \qquad \gamma \in \mathbb{R}. \tag{37}$$

By construction, $h_\ell(\gamma) = \hat{h}_\ell(\gamma/B_\ell)$, and therefore

$$|[\gamma]_+ - h_\ell(\gamma)| \le \delta_\ell, \qquad \forall \gamma \in [-B_\ell, B_\ell]. \tag{38}$$

3) Polynomial Surrogate: Based on (37), let us define

$$s_\ell(U; x_0) := \sum_{j=1}^{p} h_\ell(g_j(U; x_0)). \tag{39}$$

Based on our polynomial design assumption, our domain of approximation is as follows

$$\mathcal{D}_{B_\ell} := \{U \in \mathbb{R}^{Nm} : \|g(U; x_0)\|_\infty \le B_\ell\}. \tag{40}$$

Since $U^{(i)} \sim \tilde{\kappa}(\cdot \mid x_0)$ is Gaussian, the residual $g(U^{(i)}; x_0)$ is random and $\|g(U^{(i)}; x_0\|_\infty$ is not deterministically bounded. However, we can choose B_ℓ such the inequality is satisfied with high probability.

On $\mathcal{D}_{B_\ell}$, the total approximation error is controlled as shown in Leamma 3

Lemma 3: Assume $|[\gamma]_+ - h_\ell(\gamma)| \le \delta_\ell$ for all $\gamma \in [-B_\ell, B_\ell]$. Then for all $U \in \mathcal{D}_{B_\ell}$,

$$|s(U; x_0) - s_\ell(U; x_0)| \le p\,\delta_\ell.$$

4) Polynomial-based Desirability: Our polynomial surrogate now defines the polynomial-based desirability

$$r_\ell(U; x_0) := \exp\big(-\eta\, s_\ell(U; x_0)\big), \tag{41}$$

where $\eta > 0$ is a user-chosen sharpness parameter.

Roughly, larger violations yield larger s_ℓ and therefore smaller weights. When $s_\ell(U; x_0) \approx 0$, i.e., the trajectory is feasible, $r_\ell(U; x_0) \approx 1$.

$r(U; x_0) := \exp\big(-\eta\, s(U; x_0)\big)$,
$r_\ell(U; x_0) := \exp\big(-\eta\, s_\ell(U; x_0)\big)$,

5) Encrypted Implementation Plan:

a) Cloud-Side: The residual $g(U; x_0) = GU - h(x_0)$ is affine in U, and thus each component $g_j(U; x_0)$ can be computed homomorphically from an encrypted sample $\mathsf{Enc}(U)$ based on simple linear operations. Applying the polynomial $h_\ell(\cdot)$ to each encrypted residual component and summing over $j = 1, \dots, p$ yields $\mathsf{Enc}(s_\ell(U; x_0))$. To illustrate this more explicitly, consider the residual in relation to (27).

$$g(U^{(i)}; x_0) = GU^{(i)} - h(x_0) = Gm_U(x_0) + GL_U\xi^{(i)} - h(x_0). \tag{42}$$

Hence, we can define the state-dependent offset

$$b(x_0) := Gm_U(x_0) - h(x_0), \tag{43}$$

and an additional, state-independent (cached) perturbations

$$\xi_G^{(i)} := GL_U\xi^{(i)}. \tag{44}$$

Algorithm 1 Offline Protocol for Variational Encrypted MPC

Input: Model (A, B), horizon N, condensed MPC matrices (H, S), reference covariance $\Sigma_0 \succ 0$, temperature $\lambda > 0$, constraint matrix G, sample count K

Output: Offline cache $\{\mathsf{Enc}(L_U\xi^{(i)}), \mathsf{Enc}(\xi_G^{(i)})\}_{i=1}^K$ for the online protocol

Client (trusted): offline preprocessing

Compute $\Sigma_U := (\Sigma_0^{-1} + \frac{1}{\lambda}H)^{-1}$
Factorize $\Sigma_U = L_U L_U^\top$
Compute $\Gamma := GL_U \in \mathbb{R}^{p \times Nm}$
Encrypt and transmit $\mathsf{Enc}(L_U)$ and $\mathsf{Enc}(\Gamma)$

Cloud (untrusted): offline sampling and caching

for $j = 1, \dots, KT$ **do**
  Sample $\xi^{(j)} \sim \mathcal{N}(0, I)$ (plaintext)
  Cache $\{\mathsf{Enc}(L_U) \odot \xi^{(j)} = \mathsf{Enc}(L_U\xi^{(j)})\}$
  Cache $\{\mathsf{Enc}(\Gamma) \odot \xi^{(j)} = \mathsf{Enc}(\xi_G^{(j)})\}$

This way, we can efficiently compute the constraint residual

$$g(U^{(i)}; x_0) = b(x_0) + \xi_G, \tag{45}$$

using another cached, perturbed noise and encrypted addition.

Note: for faster implementation, we use plaintext polynomial coefficients for evaluating h_ℓ because these coefficients do not reveal sensitive data.

b) Client-Side: Since the exponential map in (41) is not polynomial, it is evaluated on the client after decryption. Concretely, for each sample $U^{(i)}$, the cloud returns

$$\Big(\mathsf{Enc}(U^{(i)}),\ \mathsf{Enc}(s_\ell(U^{(i)}; x_0))\Big), \tag{46}$$

and the client decrypts $s_\ell(U^{(i)}; x_0)$ and computes the weight

$$r_\ell^{(i)} := r_\ell(U^{(i)}; x_0) = \exp\Big(-\eta\, s_\ell(U^{(i)}; x_0)\Big). \tag{47}$$

Replacing $r(U^{(i)}; x_0)$ in (30) with the polynomial surrogate $r_\ell^{(i)}$ in (47) gives the following estimator

$$\hat{U}_{K,\ell}(x_0) := \frac{\sum_{i=1}^K U^{(i)}\, r_\ell^{(i)}}{\sum_{i=1}^K r_\ell^{(i)}}. \tag{48}$$

Estimator errors between $\hat{U}_{K,\ell}$ and $\hat{U}_K$?

G. Encrypted Variational MPC Protocol

In order to maximize computational efficiency during the execution, i.e., minimizing encrypted operatons, we decompose the protocol into an offline protocol (preprocessing phase) executed once for the given system, and an online protocol (execution phase) executed at each MPC timestep.

Algorithm 1 and 2 summarize the proposed client-cloud protocol realizing the variational encrypted MPC, with a clear separation between offline and online phases. The protocol directly instantiates the computation architecture illustrated in Figure **??**.

VI. Numerical Example

A classic double integrator example adopted from [9] is used to illusrate our protocol.

Algorithm 2 Online Protocol for Variational Encrypted MPC

Input: current state x_t
Output: control sequence $\hat{U}_K(x_t)$

Client (trusted):
Compute $m_U(x_t) = -\frac{1}{\lambda}\Sigma_U S^\top x_t$
Compute $b(x_t) = G m_U(x_t) - h(x_t)$
Send $\mathsf{Enc}(m_U(x_t))$ and $\mathsf{Enc}(b(x_t))$ to cloud
Cloud (untrusted):
for $i = 1, \dots, K$ **(parallel) do**
  Retrieve $\mathsf{cache}^{(i)} = \{\mathsf{Enc}(L_U \xi^{(i)}), \mathsf{Enc}(\xi_G^{(i)}\}_{i=1}^K$
  Compute $\mathsf{Enc}(U^{(i)}) = \mathsf{Enc}(m_U) \oplus \mathsf{cache}_0^{(i)}$
  Compute $\mathsf{Enc}(g^{(i)})) = \mathsf{Enc}(b(x_t)) \oplus \mathsf{cache}_1^{(i)}$
  Evaluate $\mathsf{Enc}(s_\ell^{(i)}) = \sum_{j=1}^{p} h_\ell\Big(\mathsf{Enc}(g_j^{(i)})\Big)$
Return $\{\mathsf{Enc}(U^{(i)}), \mathsf{Enc}(s_\ell^{(i)})\}_{i=1}^K$ to client
Client:
for $i = 1, \dots, K$ **do**
  Decrypt $U^{(i)}$ and $s_\ell^{(i)}$
  Compute $r_\ell^{(i)} = \exp(-\eta\, s_\ell^{(i)})$
$\hat{U}_{K,\ell}(x_t) = \dfrac{\sum_{i=1}^K U^{(i)} r_\ell^{(i)}}{\sum_{i=1}^K r_\ell^{(i)}}$
Apply first control input of $\hat{U}_K(x_t)$

VII. Conclusion

Appendix

A. Proofs

References

[1] H. Homburger, F. Messerer, M. Diehl, and J. Reuter, "Optimality and suboptimality of mppi control in stochastic and deterministic settings," *IEEE Control Systems Letters*, 2025.

[2] H. Homburger, S. Wirtensohn, and J. Reuter, "Mppi control of a self-balancing vehicle employing subordinated control loops," in *2023 European Control Conference (ECC)*. IEEE, 2023, pp. 1–6.

[3] Y. Zhang, C. Pezzato, E. Trevisan, C. Salmi, C. H. Corbato, and J. Alonso-Mora, "Multi-modal mppi and active inference for reactive task and motion planning," *IEEE Robotics and Automation Letters*, 2024.

[4] L. L. Yan and S. Devasia, "Output-sampled model predictive path integral control (o-mppi) for increased efficiency," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 279–14 285.

[5] B. Vlahov, J. Gibson, M. Gandhi, and E. A. Theodorou, "Mppi-generic: A cuda library for stochastic trajectory optimization," *arXiv preprint arXiv:2409.07563*, 2024.

[6] P. Halder, H. Homburger, L. Kiltz, J. Reuter, and M. Althoff, "Trajectory planning with signal temporal logic costs using deterministic path integral optimization," *arXiv preprint arXiv:2503.01476*, 2025.

[7] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 409–437.

[8] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for Approximate Homomorphic Encryption," in *Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2018, pp. 360–384.

[9] F. Borrelli, A. Bemporad, and M. Morari, *Predictive control for linear and hybrid systems*. Cambridge University Press, 2017.